
MINIGPT: REBUILDING GPT FROM FIRST PRINCIPLES

A PREPRINT

✉ **Jibin Joseph**

Department of Computer Science
The University of Texas at Austin
Austin, TX 78712
jibinjoseph@utexas.edu

May 19, 2026

ABSTRACT

This paper presents MiniGPT, a compact from-scratch implementation of GPT-style autoregressive language modeling in PyTorch. The aim is to rebuild the core GPT pipeline from first principles after studying the design of nanoGPT by Andrej Karpathy, while keeping the model and training code independently written in a single notebook. MiniGPT implements token and positional embeddings, causal multi-head self-attention, pre-LayerNorm Transformer blocks, residual connections, feed-forward MLP layers, next-token cross-entropy training (teacher forcing), validation tracking, checkpoint selection, and autoregressive text generation. This paper evaluates the implementation on Tiny Shakespeare dataset using character-level tokenization. A baseline 0.83M-parameter model reaches a validation loss of 1.7236 after 3000 training iterations. A stronger 10.77M-parameter configuration, using a larger context length and improved training settings, reaches a best validation loss of 1.4780 and generates text with recognizable Shakespeare-style dialogue structure. MiniGPT does not introduce a new language-model architecture. Instead, it documents a clear and reproducible implementation path from raw text to trained character-level generation, including design choices, training behavior, generation quality, and practical limitations.

Keywords MiniGPT · Decoder-only Transformer · Autoregressive text generation · PyTorch

1 Introduction

Generative language models have become one of the most visible outcomes of modern deep learning. Modern deep learning is often described as representation learning with multilayer neural networks [LeCun et al., 2015], while broader historical accounts show that deep learning also has a longer lineage in neural networks, credit assignment, supervised learning, unsupervised learning, and reinforcement learning [Schmidhuber, 2015]. In language modeling, this idea becomes especially important because a model can learn from raw text by predicting the next token in a sequence. The Transformer architecture made this approach more practical by using attention instead of recurrence or convolution as the main sequence-modeling mechanism [Vaswani et al., 2017]. GPT-style models build on this idea by using a decoder-only Transformer trained with an autoregressive next-token prediction objective [Radford et al., 2018, 2019]. Later large language models showed that the same basic training idea can scale to much larger models and broader language tasks [Brown et al., 2020].

However, the core GPT pipeline can still be difficult to understand from high-level explanations alone. Many descriptions show the model as a diagram, while many real implementations include engineering details that are useful for large-scale training but harder to follow at first. Compact implementations such as minGPT and nanoGPT helped make GPT-style modeling easier to inspect in code [Karpathy, 2020, 2022]. nanoGPT is especially useful as a practical reference because it organizes the model definition, training loop, evaluation, checkpointing, and sampling workflow in a small codebase [Karpathy, 2022]. Still, rebuilding the same core ideas in a single notebook can make the full path from raw text to generated samples more direct and easier to reproduce.

This paper presents MiniGPT, a compact from-scratch implementation of GPT-style autoregressive language modeling in PyTorch. MiniGPT follows the high-level decoder-only Transformer design, but the model and training code are written independently. The implementation includes token embeddings, positional embeddings, masked multi-head self-attention, feed-forward Transformer blocks, residual connections, layer normalization, next-token cross-entropy training, validation-loss tracking, checkpoint selection, and autoregressive text generation. The goal is not to introduce a new language-model architecture. Instead, the goal is to document a clear and reproducible implementation path for a small GPT-style model.

MiniGPT is evaluated on Tiny Shakespeare dataset using character-level tokenization. This setting is intentionally small, but it is useful because the dataset contains repeated structure such as speaker names, punctuation, line breaks, and dialogue-like text. The implementation first trains a baseline model to verify the complete pipeline, and then trains a stronger configuration to study the effect of increased model capacity, longer context length, validation-based checkpoint selection, and autoregressive generation quality.

The main contributions of this paper are:

- A from-scratch PyTorch implementation of a small decoder-only GPT-style language model.
- A complete character-level language-modeling pipeline from raw text to train and validation batches.
- Baseline and stronger Tiny Shakespeare dataset experiments with loss tracking and checkpoint selection.
- Qualitative generation analysis using autoregressive sampling.
- A practical discussion of model size, context length, tokenization, overfitting, and sampling behavior.

MiniGPT is a technical implementation and reproducibility paper. It does not claim architectural novelty or state-of-the-art language modeling performance. Its value is in making the main GPT-style training pipeline explicit, compact, and reproducible.

2 Background and Related Work

2.1 Transformer Language Models

Language modeling is the task of learning patterns in text so that a model can predict upcoming tokens from previous context. Earlier neural language models showed that distributed representations can reduce the sparsity problem in word-sequence modeling [Bengio et al., 2003]. The Transformer later changed sequence modeling by replacing recurrence and convolution with attention-based layers [Vaswani et al., 2017]. This made it possible to build models where each token representation is computed from other tokens in the sequence through self-attention.

GPT-style language models use the Transformer idea in a decoder-only setting. In this setup, the model reads only the previous context and learns to predict the next token. This autoregressive training objective was used in the original GPT work for generative pre-training [Radford et al., 2018]. GPT-2 showed that the same basic objective can support broader language behavior when scaled to larger models and larger text datasets [Radford et al., 2019]. GPT-3 further showed that large autoregressive language models can perform many tasks through prompting and in-context examples, without updating model parameters for each task [Brown et al., 2020].

MiniGPT follows the same general next-token prediction idea, but it keeps the setting intentionally small. Instead of focusing on scale, it focuses on making the main parts of a GPT-style model visible.

2.2 Tokenization and Small Language-Model Settings

Tokenization is an important design choice in language modeling. Many larger language models use subword or byte-level tokenization because it gives a practical balance between word-level meaning and character-level flexibility. Subword methods such as byte pair encoding are useful because they can represent rare or unseen words as smaller units [Sennrich et al., 2016]. GPT-2 also used a byte-level byte pair encoding approach as part of its language modeling pipeline [Radford et al., 2019].

MiniGPT uses character-level tokenization instead. Character-level language modeling with self-attention has also been studied in prior work, showing that deep attention-based models can learn useful structure directly from character sequences [Al-Rfou et al., 2019]. This choice is simpler than subword tokenization and keeps the full preprocessing pipeline easy to inspect. Each unique character becomes one token, so the vocabulary is small and no external tokenizer is required. The trade-off is that the model must learn words, punctuation, and longer patterns from individual characters, Which makes long-range coherence harder than with subword tokenization [Sennrich et al., 2016].

2.3 nanoGPT as a Design Reference

Several implementation-oriented resources have made GPT-style models easier to study in code. Karpathy [2020] provides a minimal PyTorch reimplementation of GPT focused on small, readable model code. Karpathy [2022] extends this direction with a compact but more practical GPT training repository. nanoGPT is especially relevant because it includes a character-level Shakespeare example, model definition, training loop, validation evaluation, checkpointing, and sampling workflow [Karpathy, 2022].

MiniGPT uses nanoGPT as a design reference, not as copied or imported code. The aim is to rebuild the same core ideas in a single notebook: a decoder-only Transformer, causal masking, next-token training, and autoregressive generation. This keeps the implementation smaller and more direct than a full training repository, while still preserving the main GPT-style workflow.

There are also larger Transformer software ecosystems, such as the Hugging Face Transformers library, which provide many pretrained models and unified APIs for training, fine-tuning, and inference [Wolf et al., 2020]. MiniGPT is different from these libraries. It is not a pretrained-model platform and it is not meant to cover many architectures. It is a compact implementation report that shows the smallest complete path from raw text to trained character-level generation.

3 MiniGPT Architecture

MiniGPT is implemented as a small decoder-only Transformer language model. The model follows the standard GPT-style structure: token IDs are converted into token embeddings, positional information is added, the sequence is processed by a stack of Transformer blocks, and a final linear language-model head produces logits over the vocabulary. This follows the general Transformer design introduced by Vaswani et al. [2017] and the decoder-only autoregressive language-modeling direction used in GPT-style models [Radford et al., 2018, 2019]. The implementation is intentionally compact and written from scratch in PyTorch, while using nanoGPT only as a high-level design reference [Karpathy, 2022].

3.1 Model Overview

The high-level architecture of MiniGPT is:

$$\text{Token IDs} \rightarrow \text{Token Embedding} + \text{Positional Embedding} \rightarrow \text{Transformer Blocks} \times L \rightarrow \text{Final LayerNorm} \rightarrow \text{Linear LM Head}$$

The model receives a batch of token IDs with shape (B, T) , where B is the batch size and T is the sequence length. Each token ID is mapped to a learned vector using a token embedding table. A learned positional embedding is also added so that the model can distinguish the order of tokens in the sequence. This is necessary because self-attention alone does not directly encode token position [Vaswani et al., 2017].

The main architecture settings are stored in a configuration object. This includes the context length, vocabulary size, number of Transformer layers, number of attention heads, embedding dimension, and dropout probability. In the baseline configuration, MiniGPT uses a context length of 128, 4 Transformer layers, 4 attention heads, and an embedding dimension of 128. This gives a small model with 826,433 trainable parameters. In the stronger configuration, MiniGPT uses a context length of 256, 6 Transformer layers, 6 attention heads, and an embedding dimension of 384, giving 10.77M trainable parameters (Table 1, Table 2)).

3.2 Causal Self-Attention

The core operation in MiniGPT is causal multi-head self-attention. For each input sequence, the model projects the hidden states into query, key, and value vectors. These vectors are split across multiple attention heads, allowing the model to learn different token-to-token relationships in parallel. The attention scores are computed using scaled dot-product attention, following the Transformer formulation [Vaswani et al., 2017].

For one attention head, this can be written as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_{\text{head}}}} + M \right) V,$$

where M is the causal mask. In MiniGPT, $M_{ij} = 0$ when $i \geq j$, and $M_{ij} = -\infty$ when $i < j$, so future positions are blocked before the softmax.

Because MiniGPT is an autoregressive language model, each token is only allowed to attend to itself and earlier tokens. It should not attend to future tokens, because the future token is the target that the model is trying to predict. MiniGPT enforces this using a lower-triangular causal mask. The mask blocks all future positions before the softmax operation. This makes the model suitable for next-token prediction, because the prediction at each position depends only on the available left context.

In implementation terms, the attention module applies the following steps: project inputs into queries, keys, and values; reshape them into multiple heads; compute scaled attention scores; apply the causal mask; apply softmax; combine the values; merge the heads; and apply a final output projection. This keeps the implementation close to the mathematical idea of self-attention and makes the masking behavior easy to inspect.

3.3 Transformer Block

Each MiniGPT Transformer block contains two main submodules: causal self-attention and a feed-forward MLP. The block uses residual connections around both submodules. Residual connections are useful because they make deeper neural networks easier to optimize by allowing layers to learn changes relative to their inputs [He et al., 2016]. In MiniGPT, the block has the following form:

$$\begin{aligned} \mathbf{x} &= \mathbf{x} + \text{Attention}(\text{LayerNorm}(\mathbf{x})) \\ \mathbf{x} &= \mathbf{x} + \text{MLP}(\text{LayerNorm}(\mathbf{x})) \end{aligned}$$

This is a pre-LayerNorm design, where layer normalization is applied before the attention and MLP submodules. Layer normalization stabilizes hidden-state statistics during training [Ba et al., 2016]. The pre-LayerNorm placement is also commonly used in practical Transformer implementations because it can make optimization more stable, especially as models become deeper [Xiong et al., 2020].

The feed-forward MLP expands the embedding dimension by a factor of four and then projects it back to the original embedding dimension. MiniGPT uses the structure:

$$\text{Linear}(d, 4d) \rightarrow \text{GELU} \rightarrow \text{Linear}(4d, d) \rightarrow \text{Dropout}$$

where d is the embedding dimension. GELU is used as the activation function because it is commonly used in Transformer language models and provides a smooth nonlinear transformation [Hendrycks and Gimpel, 2016]. Dropout is included as a regularization method, especially because Tiny Shakespeare is a small dataset and overfitting can happen easily [Srivastava et al., 2014].

3.4 Language-Model Head and Loss

After the final Transformer block, MiniGPT applies a final LayerNorm and then a linear language-model head. The language-model head maps each hidden state to a vector of logits over the vocabulary. For a character-level Tiny Shakespeare model, the vocabulary size is 65, so each position produces 65 logits.

When target tokens are provided, MiniGPT computes the cross-entropy loss between the predicted logits and the next-token targets. The target sequence is the input sequence shifted by one position. This means that at every position, the model learns to predict the next character from the previous context. This is the same basic autoregressive objective used in GPT-style language modeling [Radford et al., 2018, 2019].

In the stronger MiniGPT configuration, the token embedding matrix and language-model head weights are tied when their shapes match. Weight tying uses the same matrix for input token embeddings and output prediction weights. This can reduce the number of trainable parameters and has been shown to improve language-model performance in some settings [Press and Wolf, 2017]. Here, $V = 65$ and $d_{\text{embd}} = 384$, so tying the token embedding and language-model head weights saves $65 \times 384 = 24,960$ parameters. In MiniGPT, this is used as a simple practical improvement while keeping the architecture easy to understand.

4 Training Pipeline

MiniGPT uses a simple training pipeline for character-level autoregressive language modeling. The goal of the pipeline is to convert raw text into token sequences, form next-token prediction examples, train the model with cross-entropy loss, and track both training and validation loss over time. This follows the standard autoregressive language-modeling setup used in GPT-style models, where the model learns to predict the next token from the previous context [Radford et al., 2018, 2019].

4.1 Dataset and Tokenization

The experiments use the Tiny Shakespeare, a small text dataset commonly used for character-level language modeling examples [Karpathy, 2015]. The dataset is small enough to train quickly in a notebook environment, but it still contains useful structure such as speaker names, line breaks, punctuation, and dialogue-like text.

MiniGPT uses character-level tokenization as discussed in Section 2.2. In this setup, each unique character in the dataset is treated as one token. The tokenizer builds two dictionaries: one mapping characters to integer token IDs, and another mapping token IDs back to characters. This gives a vocabulary size of 65 characters.

After tokenization, the full text is converted into one long tensor of integer token IDs. The data is then split into training and validation sets using a 90/10 split. The training split is used for parameter updates, while the validation split is used only to estimate how well the model is doing on unseen text.

4.2 Input-Target Construction

For next-token prediction, each training example is built from a fixed-length block of tokens. Given a starting index i and a context length called `block_size`, the input sequence is:

```
x = tokens[i : i + block_size]
```

and the target sequence is the same window shifted one token to the right:

```
y = tokens[i + 1 : i + block_size + 1].
```

This means that at every position, the model receives the current and previous characters and learns to predict the next character. For example, if the input is `hell`, the target is `ello`. This shifted input-target construction is the core of autoregressive language modeling.

MiniGPT samples random starting positions from either the training split or the validation split. For each minibatch, it stacks multiple independent token blocks into tensors of shape (B, T) , where B is the batch size and T is the context length. These tensors are moved to the selected device, either CPU or GPU, before the forward pass.

4.3 Optimization and Validation

MiniGPT is trained using cross-entropy loss between the predicted vocabulary logits and the shifted target tokens. The logits are reshaped from (B, T, V) to $(B \times T, V)$, where V is the vocabulary size, and the targets are reshaped from (B, T) to $(B \times T)$. This lets the model compute one next-token prediction loss for every position in every sequence.

The baseline model uses AdamW with a fixed learning rate. AdamW is used because it applies decoupled weight decay, which is a practical improvement over standard Adam-style weight decay handling [Loshchilov and Hutter, 2019]. Adam itself is based on adaptive estimates of first and second moments of gradients, which makes it a widely used optimizer for neural network training [Kingma and Ba, 2015].

The baseline training run uses a small configuration: batch size 32, context length 128, learning rate 3×10^{-4} , and 3000 training iterations (Table 1). Periodically, MiniGPT estimates both training loss and validation loss by averaging over several minibatches. This makes the loss curve less noisy than reporting a single batch loss. The results are presented in the Section 5 (Table 2).

A stronger MiniGPT configuration is also trained to improve generation quality. This run uses a larger model, context length 256, batch size 64, dropout 0.2, AdamW with parameter groups, warmup followed by cosine learning-rate decay, gradient clipping, mixed precision training, and checkpoint selection based on validation loss. The stronger run uses a maximum learning rate of 10^{-3} , minimum learning rate of 10^{-4} , 100 warmup steps, cosine decay over 5000 steps, AdamW betas of (0.9, 0.99), weight decay of 10^{-1} , and gradient clipping at 1.0 (Table 1). For AdamW parameter grouping, tensors with dimension two or higher use weight decay, while one-dimensional tensors such as biases and LayerNorm parameters use no weight decay; in the implementation this gives 38 decayed parameter tensors and 63 non-decayed parameter tensors. Cosine-style learning-rate schedules are commonly used to reduce the learning rate smoothly during training [Loshchilov and Hutter, 2017]. Gradient clipping is used to limit unusually large gradients and improve training stability [Pascanu et al., 2013]. Mixed precision is used on GPU to reduce memory use and improve training speed while keeping training stable with loss scaling [Micikevicius et al., 2018].

Setting	Baseline MiniGPT	Stronger MiniGPT
Dataset	Tiny Shakespeare	Tiny Shakespeare
Tokenization	Character-level	Character-level
Vocabulary size	65	65
Train/validation split	90/10	90/10
Batch size	32	64
Context length	128	256
Training iterations	3000	5000
Optimizer	AdamW	AdamW
Learning-rate schedule	Fixed	Warmup + cosine decay
Gradient clipping	No	Yes
Mixed precision	No	Yes, on CUDA
Checkpoint selection	Final model	Best validation loss
Maximum learning rate	3×10^{-4}	10^{-3}
Minimum learning rate	–	10^{-4}
Warmup steps	–	100
Cosine decay steps	–	5000
AdamW betas	Default	(0.9, 0.99)
Weight decay	Default AdamW setting	10^{-1} for decayed parameters
Parameter grouping	No	38 decayed / 63 non-decayed tensors

Table 1: Summary of the MiniGPT training pipeline settings.

5 Experiments and Results

This section reports two MiniGPT experiments on the Tiny Shakespeare dataset as discussed in Section 4.3. The first experiment uses a small baseline model to verify that the full training pipeline works correctly. The second experiment uses a stronger model configuration to improve validation loss and generation quality. The goal is not to reach state-of-the-art language modeling performance, but to show that the implementation learns meaningful structure from raw text.

5.1 Baseline Configuration

The baseline experiment uses a small decoder-only Transformer with 4 layers, 4 attention heads, an embedding dimension of 128, and a context length of 128 characters. This model has 826,433 trainable parameters. It is trained for 3000 iterations using AdamW and character-level next-token prediction. AdamW is used because it is a standard optimizer for Transformer training and applies decoupled weight decay [Loshchilov and Hutter, 2019].

At the start of training, both training and validation loss are close to 4.20. This is expected because the model starts with random weights and the Tiny Shakespeare vocabulary has 65 characters. During training, both losses decrease steadily. By step 3000, the baseline model reaches a training loss of 1.5304 and a validation loss of 1.7236 (Table 2, Figure 2). This confirms that the model learns useful local patterns from the text and that the training loop, validation loop, and loss computation are working correctly.

5.2 Stronger Configuration

The stronger experiment increases the model capacity and training quality. It uses 6 Transformer layers, 6 attention heads, an embedding dimension of 384, and a context length of 256 characters. This gives a 10.77M-parameter model. The stronger configuration also uses dropout, AdamW parameter groups, warmup followed by cosine learning-rate decay, gradient clipping, mixed precision training on CUDA, and checkpoint selection based on validation loss.

This configuration is close to the small Shakespeare character-level setup used in nanoGPT, where a 6-layer, 6-head, 384-channel GPT is trained on Tiny Shakespeare dataset [Karpathy, 2022]. MiniGPT does not reuse nanoGPT code, but this comparison is useful because it gives a practical reference point for the scale of the experiment.

The stronger model starts with a validation loss of 4.2879. The loss then drops quickly during the early part of training. The best validation loss is 1.4780 at step 1750. The corresponding validation perplexities are approximately 5.60 for the baseline model and 4.38 for the stronger best checkpoint (Table 2). After this point, the training loss continues to decrease, but the validation loss begins to increase (Figure 2). This shows that the model starts to overfit the training text after the best checkpoint. Validation-based checkpoint selection is therefore important, because the final model at step 5000 is not the best model for generalization [Prechelt, 1998].

Metric	Baseline MiniGPT	Stronger MiniGPT
Parameters	826,433	10.77M
Layers / heads / embedding dim	4 / 4 / 128	6 / 6 / 384
Context length	128	256
Batch size	32	64
Training iterations	3000	5000
Checkpoint used	Final step	Best validation checkpoint
Best/final training loss	1.5304	1.0990 at step 1750
Best/final validation loss	1.7236 at step 3000	1.4780 at step 1750
Validation perplexity, $\exp(\text{loss})$	5.60	4.38
Final validation loss	1.7236	1.7055 at step 5000
Training time	50.79 seconds	4.76 minutes

Table 2: Comparison of baseline and stronger MiniGPT experiments on Tiny Shakespeare dataset. Perplexity is computed as $\exp(\text{validation loss})$ for the reported validation checkpoint.

5.3 Loss Curves and Checkpoint Selection

The loss curves show two important behaviors. First, the stronger model learns faster and reaches a lower validation loss than the baseline model. The baseline model reaches 1.7236 validation loss after 3000 steps (Figure 1), while the stronger model reaches 1.4780 validation loss by step 1750 (Figure 2). This improvement is mainly due to the larger model capacity, longer context length, stronger regularization, learning-rate scheduling, and better checkpoint selection.

Second, the stronger model clearly shows overfitting after step 1750. At step 1750, the training loss is 1.0990 and the validation loss is 1.4780. By step 5000, the training loss has decreased further to 0.6105, but the validation loss has increased to 1.7055 (Table 2, Figure 2). This means the model is fitting the training set more strongly, but it is no longer improving on unseen validation text. For this reason, the best checkpoint is selected using the lowest validation loss, not the final training step.

The experiments show that MiniGPT successfully learns from the Tiny Shakespeare dataset. The baseline model is enough to verify the full training pipeline, while the stronger model gives a clear improvement in validation loss and produces better text samples. The results also show a practical limitation: increasing model capacity and training time improves performance only up to a point. After the best validation checkpoint, more training causes overfitting rather than better generalization.

6 Qualitative Generation Analysis

6.1 Generated Samples

After training, MiniGPT is evaluated qualitatively by generating text from short prompts. This is useful because validation loss shows whether the model predicts unseen text better, but generated samples show what the model has actually learned in a more readable way. Open-ended language models are often inspected using generated samples, especially when the goal is to understand the behavior of autoregressive generation [Radford et al., 2019]. Decoding choices are also important, because the same trained model can produce different quality text under different sampling settings [Holtzman et al., 2020].

The stronger MiniGPT checkpoint is used for this analysis because it gives the best validation loss. The samples are generated with `max_new_tokens=800`, temperature 0.8, and `top_k=200`. Text is generated autoregressively: the model receives a prompt, predicts the next character, samples from the predicted distribution, appends the sampled character to the context, and repeats this process until the requested number of new characters is produced.

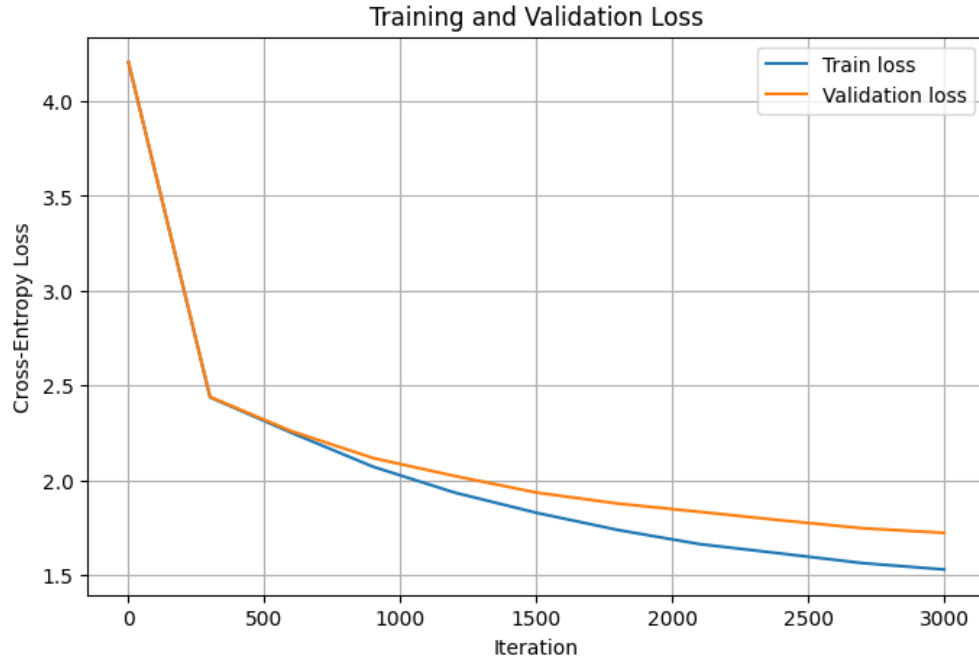


Figure 1: Training and validation loss for the baseline MiniGPT model on Tiny Shakespeare dataset. Both losses decrease steadily from about 4.20 at step 0 to 1.5304 training loss and 1.7236 validation loss at step 3000, showing that the baseline model learns useful structure from the dataset without clear overfitting.

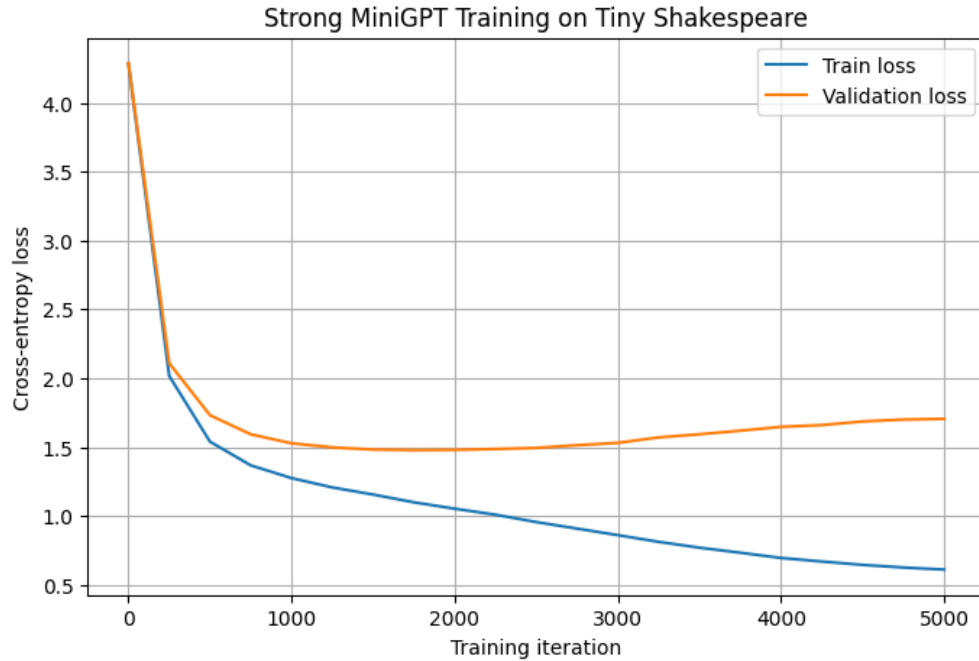


Figure 2: Training and validation loss for the stronger MiniGPT model on Tiny Shakespeare dataset. The validation loss reaches its minimum value of 1.4780 at step 1750, while the training loss continues to decrease afterward. By step 5000, the training loss falls to 0.6105, but the validation loss increases to 1.7055, indicating overfitting after the best validation checkpoint (step 1750).

The following excerpts show representative generated samples. The samples are shortened for space. The full generated samples are given in the MiniGPT repo Joseph [2026].

Prompt: ROMEO:
 ROMEO:
 Nay, fie, I'll plead it, I have the indeed,
 Go to the deed! I think it was a back!
 BRUTUS:
 So did let us continue them home.
 SICINIUS:
 Only, brawling not
 The common 'twixt him, where enter'd his eyes...

Prompt: \n (newline character)
 She which she hath death seen his prey daughter:
 It is too much not leads; but to know the right.
 PARIS:
 No, by God's deed, but I fear the sea
 Which would have the strange for the degree.
 KING RICHARD III:
 Ay, have I been my mother's deed...

The generated text is not fully meaningful, but it clearly learns some visible structure from Tiny Shakespeare dataset. It produces speaker names, line breaks, punctuation, and dialogue-like formatting. It also creates short phrases that look similar to the training dataset. This shows that the model has learned local character patterns and some formatting regularities from the dataset.

6.2 Temperature and Top-k Sampling

MiniGPT uses sampling rather than always selecting the most likely next character. During generation, the logits from the final time step are divided by a temperature value before applying softmax. A lower temperature makes the probability distribution sharper, so the model chooses high-probability characters more often. A higher temperature makes the distribution flatter, so the output becomes more varied but also less stable.

In this experiment, temperature has a clear effect on generation quality. With temperature 0.7, the samples are more stable and readable. The model keeps better dialogue structure and produces fewer strange words, but the output can become more repetitive. With temperature 1.2, the samples are more diverse and creative, but they also contain more broken spellings and unusual words such as invented names or malformed phrases. This behavior is expected because decoding strategies can strongly affect the quality and diversity of language-model output [Holtzman et al., 2020].

MiniGPT also supports top-k sampling in the generation function. Top-k sampling keeps only the k most likely next tokens before sampling. This type of sampling was also used in GPT-2 generation experiments [Radford et al., 2019]. However, in this MiniGPT run, the vocabulary size is only 65 characters and the generation setting uses `top_k=200`. Since 200 is larger than the vocabulary size, top-k filtering does not remove any character in this specific experiment. Therefore, the main sampling factor in this experiment is temperature. The top-k option is still useful in the implementation because it makes the generation code general and easy to extend to larger vocabularies.

Setting	Observed behavior	Main trade-off
Temperature = 0.7	More stable and readable	Less diverse, more repetitive
Temperature = 0.8	Balanced sample quality	Used for main samples
Temperature = 1.2	More varied and creative	Less coherent, more strange words
Top-k = 200	No truncation for 65-character vocabulary	Kept for generality

Table 3: Qualitative effect of sampling settings in MiniGPT generation.

6.3 Observed Strengths and Failure Modes

The main strength of the generated text is that it captures local style. MiniGPT learns to generate uppercase speaker names, colon-based dialogue markers, line breaks, punctuation, and short Shakespeare-like phrases. This is a useful

result for a small character-level model, because the model is not given words directly. It has to learn spelling, word boundaries, punctuation, and formatting from characters alone.

The main failure mode is long-range meaning. The generated samples often look correct locally, but the meaning is not consistent over longer spans. Some sentences are grammatically weak, some phrases do not connect logically, and the model sometimes invents strange words. This is expected for a small character-level model trained on a narrow dataset. Character-level tokenization keeps the implementation simple, but it also makes sequences longer and makes word-level coherence harder than with subword tokenization.

The samples also show the effect of model capacity and training behavior. The stronger model generates better text than the baseline because it uses more layers, more attention heads, a larger embedding dimension, and a longer context length. At the same time, the loss curves show that training longer is not always better as discussed in Section 5.3. The strongest validation checkpoint occurs before the final training step, and later training causes overfitting. Therefore, the best generated samples should come from the checkpoint selected by validation loss, not simply from the final model.

The qualitative analysis supports the quantitative results. MiniGPT does not generate fully coherent Shakespeare-style writing, but it learns recognizable local structure from the dataset. The model demonstrates the complete GPT-style workflow: character-level tokenization, next-token prediction, validation-based checkpoint selection, and autoregressive generation.

7 Discussion and Limitations

MiniGPT shows that the core GPT-style language-modeling pipeline can be rebuilt in a compact and readable PyTorch implementation. The main contribution is the complete end-to-end reconstruction of a small decoder-only language model, from raw text processing to validation-based checkpoint selection and autoregressive generation.

The first limitation is scale. MiniGPT is trained on Tiny Shakespeare dataset with a small character-level vocabulary and a relatively small model. Scaling-law studies show that language-model loss depends strongly on model size, dataset size, and training compute [Kaplan et al., 2020]. Later work on compute-optimal training also shows that model size and training data should be scaled together under a fixed compute budget [Hoffmann et al., 2022]. MiniGPT is therefore best understood as a compact implementation study, not as a competitive large-scale language model.

The second limitation is tokenization. Character-level modeling makes sequences longer and forces the model to learn words, spelling, and punctuation from individual characters. Subword tokenization methods, such as byte pair encoding, are often more efficient for larger language-model settings because they represent rare and common word pieces using a manageable vocabulary [Sennrich et al., 2016]. In the stronger configuration, the context length is 256 characters (Table 1). Because MiniGPT uses character-level tokenization, this covers only a short passage, roughly a few dozen words, which limits long-range coherence, speaker consistency, and plot-level tracking.

The third limitation is dataset scope. Tiny Shakespeare dataset is useful because it is small, structured, and easy to train on in a notebook environment. At the same time, it is narrow in style and content. A model trained only on this dataset learns Shakespeare-like formatting, speaker names, punctuation, and local phrase patterns, but it does not learn broad language ability. The generated samples show recognizable local structure, but the meaning is often weak over longer passages.

The fourth limitation is overfitting. The stronger MiniGPT model reaches its best validation loss at step 1750 (Figure 2). After that point, training loss continues to decrease, but validation loss increases. This shows that the model starts to memorize the training text more strongly instead of improving on unseen validation text. For this reason, the best model is selected using validation-based early stopping and checkpoint selection [Prechelt, 1998].

The fifth limitation is evaluation. This paper reports training loss, validation loss, and qualitative generated samples. These are useful for checking whether the implementation works and whether the model learns meaningful structure. However, the paper does not include large-scale benchmark evaluation, human evaluation, or comparison against pretrained models. The generated samples are therefore used only as qualitative evidence, not as a claim of strong language ability.

Finally, generation quality depends on sampling choices. Lower temperature gives more stable text, while higher temperature gives more diverse but less coherent text. This behavior is expected because decoding choices can strongly affect the quality, diversity, and repetition patterns of neural text generation [Holtzman et al., 2020]. In MiniGPT, this effect is visible even in a small character-level setting.

MiniGPT should be viewed as a reproducible implementation report. Its value is in making the GPT-style training pipeline clear and complete, from raw text to generated samples. Its limitations come mainly from the intentionally small dataset, character-level tokenization, limited model scale, and qualitative evaluation setup.

8 Code Availability and Reproducibility

The MiniGPT code is publicly available at <https://github.com/jibin10/MiniGPT>. The repository includes `MiniGPT_Notebook.ipynb`, README instructions, a license file, and the details needed to run the experiment in Google Colab.

The implementation is written in Python using PyTorch [Paszke et al., 2019]. The main experiment is provided as a Jupyter notebook, so the model code, training loop, results, and generated samples can be inspected and rerun in one place [Kluyver et al., 2016]. The experiment uses the Tiny Shakespeare dataset with character-level tokenization [Karpathy, 2015].

To reproduce the results, users can open `MiniGPT_Notebook.ipynb` in Colab, select a GPU runtime, and run the notebook cells from top to bottom. The reported training times in this paper were measured on a Colab A100 GPU. The notebook sets `torch.manual_seed(42)`. Small numerical differences may still occur because of hardware, PyTorch/CUDA versions, GPU nondeterminism, and random minibatch sampling.

9 Conclusion

This paper presented MiniGPT, a compact from-scratch implementation of a GPT-style autoregressive language model in PyTorch. The implementation reconstructs the end-to-end decoder-only language-modeling pipeline, from raw text processing to validation-based checkpoint selection and autoregressive generation.

On Tiny Shakespeare dataset, the baseline MiniGPT model verified that the full training pipeline works, reaching a validation loss of 1.7236 after 3000 iterations. The stronger 10.77M-parameter configuration improved the best validation loss to 1.4780 (Table 2) and generated text with recognizable Shakespeare-style local structure, including speaker names, line breaks, punctuation, and dialogue-like formatting.

MiniGPT does not propose a new Transformer architecture or a state-of-the-art language model. Its contribution is the clear and reproducible reconstruction of the GPT-style training pipeline from raw text to generated samples. The results show that even a small character-level model can learn meaningful structure when the architecture, training loop, validation process, and sampling method are implemented carefully.

Declarations

Funding

No funding was received to assist with the preparation of this manuscript.

Competing interests

The author has no relevant financial or non-financial interests to disclose.

Data availability

The data used in this study are publicly available through the Tiny Shakespeare dataset used in Karpathy’s character-level language modeling examples [Karpathy, 2015].

Code availability

The code is available at <https://github.com/jibin10/MiniGPT>.

References

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi:10.1038/nature14539.

- Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015. doi:10.1016/j.neunet.2014.09.003.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI technical report*, 2018. URL https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI technical report*, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- Andrej Karpathy. mingpt: A minimal pytorch re-implementation of gpt. <https://github.com/karpathy/minGPT>, 2020. GitHub repository.
- Andrej Karpathy. nanogpt: The simplest, fastest repository for training/finetuning medium-sized gpts. <https://github.com/karpathy/nanoGPT>, 2022. GitHub repository.
- Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016. doi:10.18653/v1/P16-1162.
- Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. Character-level language modeling with deeper self-attention. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(1):3159–3166, 2019. doi:10.1609/aaai.v33i01.33013159.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020. doi:10.18653/v1/2020.emnlp-demos.6.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi:10.1109/CVPR.2016.90.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, pages 10524–10533. PMLR, 2020.
- Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL <https://jmlr.org/papers/v15/srivastava14a.html>.
- Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, pages 157–163. Association for Computational Linguistics, 2017. doi:10.18653/v1/E17-2025.
- Andrej Karpathy. char-rnn: Multi-layer recurrent neural networks for character-level language models. <https://github.com/karpathy/char-rnn>, 2015. GitHub repository.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.

- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Skq89Scxx>.
- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning*, pages 1310–1318. PMLR, 2013. URL <https://proceedings.mlr.press/v28/pascanu13.html>.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=r1gs9JgRZ>.
- Lutz Prechelt. Early stopping—but when? In Genevieve B. Orr and Klaus-Robert Müller, editors, *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer, 1998. doi:10.1007/3-540-49430-8_3.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rygGQyrFvH>.
- Jibin Joseph. Minigpt: Rebuilding gpt from first principles. <https://github.com/jibin10/MiniGPT>, 2026. GitHub repository.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022. URL <https://arxiv.org/abs/2203.15556>.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.
- Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks—a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016. doi:10.3233/978-1-61499-649-1-87.